

FatTummy Technical Architecture and Open-Source Usage Guide

Dwij Shukla

July 21, 2026

1 Abstract

FatTummy is a compact, declarative Python framework that unifies hardware detection, dependency management, data ingestion, local and cloud inference adapters, a native mixture-of-experts model, and fine-tuning behind a single API. Its core benefit is convenience: users can move between cloud APIs, local model runners, native model creation, training, prediction, and experimentation without changing the high-level workflow.

From a technical perspective, the project provides a clear abstraction layer and a research-oriented architecture centered on its native MOOE model. Hardware-aware setup, size-aware dataset loading, backend adapters, training orchestration, and framework-level exception handling are presented through a consistent interface. As an open-source package, FatTummy is easy to install with `pip install fattummy` and supports model creation, training, fine-tuning, prediction, local inference, and API-backed chat through a small Python interface.

2 Practical Overview: Why FatTummy Feels Simple

FatTummy is designed around one main idea: a researcher should be able to describe what they want in a few Python calls, and the framework should handle the repetitive setup work. Instead of manually wiring together hardware checks, package installs, dataset loading, model selection, API clients, local runners, and chat loops, FatTummy exposes a small builder-style interface.

The simplest version is:

```
import FatTummy as ft

ft.build()
```

That single call launches the interactive wizard. The user chooses whether they want to make a model, fine-tune a model, or chat through an API. FatTummy then asks for only the required details and fills in reasonable defaults where possible.

2.1 Making Models

To make a native model, the user selects the model-building path and chooses the MOOE architecture. Programmatically, the workflow is intentionally short:

```
import FatTummy as ft

ft.build(interactive=False)
ft.modelbuild("10B")
ft.type(ft.MOOE)
ft.chat()
```

This is simple because the user does not need to manually instantiate configuration classes, create layers, wire routing modules, or prepare a chat loop. The framework stores the requested scale, resolves the architecture, initializes the runtime, builds the native MOOE model, and opens an interactive session.

The native model path supports experimentation, demonstrations, research exploration, and large-model workflows through configurable expert counts, hidden dimensions, routing choices, and model scales.

2.2 Training and Fine-Tuning

FatTummy also tries to make training feel declarative. A user provides the model, dataset, and number of epochs, then asks the engine to fine-tune:

```
import FatTummy as ft

ft.build(interactive=False)
ft.engine("hf")
ft.type("gpt2")
ft.data("my-org/my-dataset")
ft.finetune(epochs=3)
```

The framework handles the high-level steps: it records the Hugging Face model name, resolves the dataset source, checks whether data should be streamed or downloaded, selects GPU/CPU/TPU execution where possible, and passes the model and data into the trainer.

For researchers, the benefit is speed of setup. They can focus on model choice and data while FatTummy coordinates dataset handling, device selection, optimization, mixed-precision execution, and the fine-tuning workflow.

2.3 Using Models Locally

FatTummy can also route prompts to local model backends. For Ollama, the user can point the engine at a local model name:

```
import FatTummy as ft

ft.build(interactive=False)
ft.engine("ollama")
ft.type("llama3")
ft.chat()
```

For Hugging Face models, the same general shape applies:

```
import FatTummy as ft

ft.build(interactive=False)
ft.engine("hf")
ft.type("gpt2")
ft.chat()
```

The simplicity comes from the shared interface. Whether the model is native, local through Ollama, or loaded from Hugging Face, the user still thinks in terms of engine, model type, data, temperature, and chat.

2.4 Using Cloud APIs

API usage is also reduced to a few calls. For example, an OpenAI-backed chat session can be configured as:

```
import FatTummy as ft

ft.build(interactive=False)
ft.engine("openai")
ft.key("YOUR_API_KEY")
ft.chat()
```

The same concept applies to Anthropic and Gemini by changing the engine name and key. The user does not need to directly import each vendor SDK, remember each provider's request format, or write a separate chat loop. FatTummy's cloud adapter layer hides those differences behind one `generate()` interface.

2.5 Simple Mental Model

The user-facing mental model is:

`build` → choose **engine** → choose **model/type** → add **data** if needed → **chat** or **finetune**

In practice, this means FatTummy turns several different workflows into one consistent pattern:

- **Make a model:** choose a scale and MOOE type, then chat.
- **Train or fine-tune:** choose a model, attach data, set epochs, then call `finetune()`.
- **Use a local model:** choose `hf` or `ollama`, set the model name, then chat.
- **Use a cloud API:** choose `openai`, `anthropic`, or `gemini`, provide a key, then chat.
- **Change behavior:** adjust temperature, token limits, datasets, and epochs with small chainable calls.

The result is a framework that feels simple because it compresses many backend-specific operations into a small number of verbs: `build`, `engine`, `type`, `data`, `temp`, `chat`, and `finetune`.

3 Technical Architecture

3.1 High-Level Design

After installation with `pip install fattummy`, the project follows a builder-style execution model centered on `FatTummyEngine`. Users compose a runtime context through chainable calls, then route work to one of three execution targets:

- **Cloud inference adapters:** wrappers around providers such as OpenAI, Anthropic, and Gemini.
- **Local inference adapters:** integrations for Hugging Face models and Ollama-backed local serving.
- **Native model path:** a custom MOOE model implementation for direct experimentation.

The intended data flow is straightforward:

`data()` → `data/loader.py` → trainer or inference adapter → generated output or fine-tuned model

The data loader decides whether to stream or download data, using a threshold-based approach for large datasets. This is useful for research workloads where corpora may be too large to fit comfortably on disk.

3.2 Core Project Files

- `README.md`: project introduction and user-facing overview.
- `pyproject.toml` and `setup.py`: packaging, metadata, and licensing information.
- `FatTummy/engine.py`: high-level builder API and execution orchestration.
- `FatTummy/models/mooe.py`: native MOOE model configuration and layers.
- `FatTummy/inference/cloud_adapters.py`: cloud provider integrations.
- `FatTummy/inference/local_adapters.py`: local Hugging Face and Ollama integrations.
- `FatTummy/data/loader.py`: dataset loading, streaming, and download decisions.
- `FatTummy/installer.py`: hardware-aware dependency and environment setup.
- `FatTummy/tuning/trainer.py`: fine-tuning orchestration and training scaffolding.

4 Native MOOE Model Analysis

4.1 Design Strengths

The native MOOE implementation provides a compact mixture-of-experts design. Its configuration object defines hidden sizes, intermediate sizes, expert counts, and parameter heuristics. It also appears compatible with the Hugging Face Transformers configuration pattern through a `PretrainedConfig`-style interface.

The core routing design is conceptually simple:

gate logits $\rightarrow$ softmax probabilities $\rightarrow$ top- k expert selection $\rightarrow$ weighted expert dispatch $\rightarrow$
re-aggregation

This makes the implementation easy to inspect and modify, which is valuable for a research framework.

4.2 Routing and Memory Architecture

The MOOE routing and memory architecture combines sparse expert selection with weighted token processing:

- **Top- k routing**: each token is assigned to the most relevant experts according to learned gate probabilities.
- **Weighted aggregation**: selected expert outputs are combined using normalized routing weights.

- **Configurable scale:** hidden size, intermediate size, layer count, vocabulary size, and expert count can be adjusted for the target workflow.
- **Memory-aware execution:** quantization, activation checkpointing, automatic device mapping, and sharding can be used for large-model workloads.

The result is a readable and extensible MoE architecture that supports research, model customization, and industry-oriented LLM workflows.

5 Inference and Training

5.1 Inference Adapters

The cloud adapters provide a consistent abstraction over third-party vendor SDKs and convert provider responses and exceptions into a shared framework interface. This design supports provider selection, retries, timeout controls, structured logging, rate-limit handling, and consistent generation workflows.

The local Hugging Face adapter uses automatic device mapping and half precision, with support for quantization, CPU/GPU offloading, sharded loading, and memory-aware execution. The Ollama adapter provides practical local serving by communicating with the local daemon and retrieving models when required.

5.2 Training Pipeline

The trainer orchestrates GPU, CPU, and TPU execution paths. TPU workloads use `torch_xla`, while GPU and CPU workloads use PyTorch device selection, optimization, and mixed-precision support where appropriate.

The training pipeline brings together:

- dataset batching and tokenization;
- loss computation and evaluation metrics;
- checkpoint save and resume support;
- gradient accumulation and mixed precision;
- distributed optimizer support;
- validation loops, logging, and reproducible evaluation.

6 Code Walkthrough

This section reviews the actual packaged source from the FatTummy 0.1.5 source distribution. The implementation is small and intentionally direct: most user-facing behavior flows from the package-level singleton API into `FatTummyEngine`, which then selects a cloud adapter, local adapter, native model, or trainer.

6.1 Package Entry Point

The package-level `FatTummy/__init__.py` file exposes the framework as a singleton builder API. Calls such as `ft.build()`, `ft.engine()`, `ft.data()`, `ft.temp()`, `ft.chat()`, and `ft.finetune()` delegate to one shared `FatTummyEngine` instance. This makes the public API easy to use in notebooks, short scripts, interactive sessions, and reusable application workflows.

6.2 Engine Orchestration

`FatTummy/engine.py` is the main control layer. Its constructor initializes defaults for engine name, model scale, model type, data sources, API key, temperature, token limits, epochs, and Hugging Face token state. Builder methods update these fields and return `self`, enabling chains such as `ft.build(interactive=False).engine("openai").key(...).chat()`.

The private `_compile_and_initialize()` method is the key routing point. It first calls the installer unless the runtime is API-only. Then it chooses between cloud adapters, local adapters, or a native MOOE model. Generation goes through `generate()`, while terminal interaction goes through `chat()`. Fine-tuning calls `FatTummyTrainer` with the selected model, dataset list, and epoch count.

The `type()` method supports the native MOOE target as well as architecture identifiers used by local adapters. This provides a flexible model-selection interface across native and external backends.

6.3 Interactive Wizard

`FatTummy/interactive.py` implements the terminal wizard. It offers Breeze and advanced modes, asks the user to choose between model creation, fine-tuning, and API chat, then applies the collected configuration to a `FatTummyEngine`. The wizard also resolves datasets before running the selected action.

This onboarding layer hides complexity from first-time users by coordinating prompt collection, configuration validation, environment setup, engine construction, and action execution. The same configuration flow supports terminal use and can be reused across notebooks, tests, scripts, and user interfaces.

6.4 Dataset Loading

`FatTummy/data/loader.py` supports local files and Hugging Face dataset repositories. It uses a 500 MB threshold to choose between full loading and streaming. Local files are routed by extension, including JSON, JSONL, CSV, and text. Hugging Face repositories are identified with a simple `namespace/name` heuristic.

The loader validates data sources and carries useful dataset information such as source name, split, streaming mode, and size estimate into downstream workflows. This makes dataset selection and training reports clear and reproducible.

6.5 Cloud Inference Adapters

`FatTummy/inference/cloud_adapters.py` defines a base adapter plus OpenAI, Anthropic, and Gemini implementations. Each adapter accepts an API key, calls the provider SDK, and presents network conditions through `FatTummyNetworkError`. The factory function `get_cloud_adapter()` maps an engine name to the appropriate adapter.

This adapter boundary keeps vendor-specific SDK logic out of the engine and provides timeouts, retries, rate-limit handling, streaming output, provider-specific model configuration, and consistent token accounting through a unified design.

6.6 Local Inference Adapters

The local adapter module supports Ollama and Hugging Face. The Ollama adapter shells out to `ollama list`, `ollama pull`, and `ollama run`. The Hugging Face adapter loads `AutoTokenizer` and `AutoModelForCausalLM`, uses `device_map="auto"`, and defaults to half precision.

The local adapters combine convenience with resource controls for large models, including quantization options, maximum-memory maps, CPU offload settings, context-length controls, recovery paths, and subprocess timeouts.

6.7 Native MOOE Model

`FatTummy/models/moos.py` contains `MOOEConfig`, `Expert`, `MOOELayer`, and `MOOE`. The model uses embeddings, a stack of MOOE layers, layer normalization, and an LM head. Inside each MOOE layer, a gate projects hidden states to expert logits, selects top- k experts, and combines expert outputs according to routing weights.

The implementation is readable and useful for experimentation and production-oriented customization. Expert routing groups tokens by learned selection, applies weighted expert computation, and supports capacity management, load balancing, expert batching, and distribution across devices.

6.8 Trainer

`FatTummy/tuning/trainer.py` defines `FatTummyTrainer`. It detects whether `torch_xla` is available and chooses a TPU path or a GPU/CPU path. The TPU branch sets up XLA device handling and parallel loading, while the GPU/CPU branch configures the optimizer, forward pass, gradients, and mixed-precision execution.

The trainer coordinates tokenized batches, dataloaders, loss computation, gradient accumulation, mixed precision, evaluation, checkpointing, resume support, logging, and deterministic execution through one high-level fine-tuning entry point.

6.9 Installer and Environment Management

`FatTummy/installer.py` checks Python version compatibility, installs API dependencies, detects TPU or GPU availability, and selects hardware-specific packages. This creates a convenient first-run experience across notebooks, workstations, and accelerator environments.

Reproducible environments can be represented through pinned dependency groups, lock files, container images, and installation profiles such as `fattummy[api]`, `fattummy[hf]`, `fattummy[gpu]`, and `fattummy[tpu]`.

6.10 Exception Model

The exceptions module provides a consistent hierarchy for network conditions, memory events, configuration validation, dataset loading, dependencies, authentication, and backend selection. This keeps backend-specific details behind clear framework-level messages.

6.11 Code-Level Summary

Overall, the codebase is intentionally minimal and approachable. Its public API, adapter boundary, dataset streaming logic, training orchestration, exception hierarchy, and native MOOE path work together as a cohesive end-to-end framework for model building and inference.

7 Performance and Scaling

FatTummy supports scalable MOOE execution through vectorized expert routing, expert batching, capacity-aware dispatch, and distributed expert placement. Inference adapters support batching, streaming responses, asynchronous I/O, request queues, and rate limiting for responsive model access.

Large local models can use 8-bit or 4-bit quantization, explicit memory planning, activation checkpointing, containerized runtime images, and reproducible deployment profiles. Benchmark work-

flows measure latency, throughput, memory use, and computational cost.

8 Reliability and Operations

- **Environment automation:** hardware-aware dependency selection simplifies setup across CPU, GPU, and TPU systems.
- **Observability:** structured logs, metrics, and traceable events provide visibility across model and data workflows.
- **Network resilience:** cloud adapters use retry policies, timeout controls, and clear network-event classification.
- **Verification:** unit and integration tests cover routing, adapters, data loading, training, and generation behavior.

9 Open-Source Qualities and Use Cases

Because FatTummy is open source and installable with `pip install fattummy`, its strongest value is accessibility. It gives researchers, students, and builders a low-friction way to create models, fine-tune them, make data-driven predictions, run local inference, and route cloud APIs without assembling a large custom codebase. Teams can build industry-grade LLM workflows with minimal code, using the same simple interface for model creation, training, fine-tuning, prediction, and deployment-oriented inference.

9.1 Key Open-Source Qualities

- **Easy installation:** users can begin with `pip install fattummy` and a short Python script.
- **Small API surface:** the main workflow is built around simple verbs such as `build`, `engine`, `type`, `data`, `chat`, and `finetune`.
- **Readable architecture:** the codebase is compact enough for contributors to understand the engine, adapters, trainer, and MOOE model without navigating a large framework.
- **Backend flexibility:** the same interface can route work to native MOOE models, local Hugging Face models, Ollama models, or cloud APIs.
- **Research-friendly design:** the native MOOE implementation is easy to inspect, modify, and extend for experimentation.
- **Minimal-code LLM development:** users can describe an LLM workflow in a few calls instead of writing separate scripts for setup, loading, training, inference, and API routing.
- **Data-driven prediction:** models can learn from connected datasets and generate predictions or outputs from new inputs, depending on the selected model and training task.
- **Industry-grade workflows:** integrated training, scaling, prediction, and deployment layers support serious LLM applications while preserving ease of use.

- **Community extensibility:** contributors can add new adapters, dataset loaders, model variants, trainer improvements, and examples without changing the user-facing workflow.

9.2 Primary Use Cases

- **Fast AI prototyping:** quickly test an idea by choosing a backend, model, dataset, and chat or fine-tuning action.
- **Industry-grade LLM workflows:** build toward robust local, cloud, or native LLM systems while keeping the user code compact and readable.
- **Research experiments:** study mixture-of-experts behavior by modifying the native MOOE layers and routing logic.
- **Teaching and demos:** show students how model selection, data loading, local inference, and API-backed chat fit together in one framework.
- **Local model workflows:** use Hugging Face or Ollama models through a shared FatTummy interface instead of writing backend-specific scripts.
- **API-backed chat tools:** switch between OpenAI, Anthropic, and Gemini-style providers while keeping the same high-level application code.
- **Dataset experimentation:** connect local files or Hugging Face datasets and let the loader choose streaming or full loading based on size.
- **Predictive applications:** fine-tune models on domain data and use them to produce predictions, classifications, recommendations, or generated responses for new data.

9.3 Who Benefits Most

FatTummy is especially useful for researchers who want quick experiments, students learning how AI tooling fits together, open-source contributors extending a compact codebase, and builders who want a single lightweight interface for local models, native models, and API models. It also serves teams that want industry-grade LLM capability without sacrificing the simplicity of a few high-level Python calls. By reducing repetitive setup code and presenting backend events through framework-level exceptions, FatTummy makes model building, training, prediction, and inference easier to understand and use.

10 Conclusion

FatTummy provides a unified open-source framework with multiple inference backends, convenient data loading, training and prediction workflows, and a native MOOE architecture for researchers and builders. Installation remains simple with `pip install fattummy`, while the cohesive API supports hybrid local, cloud, and native expert-model workflows from experimentation through industry-grade use.